

MLB Scion — Training Run Strategy

Long single run vs. batching, and how to select the ensemble honestly

Companion to lgbmBinaryCLASSIFIER V26.07 · 25 June 2026

1. Bottom line

Two things were tangled together in the question — *how often the run finishes and ranks* (a cadence problem) and *how you choose which models make the ensemble* (a selection problem). The cadence problem is now largely solved in code without batching; the selection problem is the one that actually matters for ROI, and it needs changing regardless of cadence.

- **Cadence.** You no longer need to batch just to see results. The log now writes **one row per model as it completes** and each epoch is logged live to MLflow, so a 1,000-model run is monitorable from hour one and is crash-safe. Keep the single long run.
- **Selection.** Stop selecting models on the outsample — both your current “precision and recall > 0.5 on the outsample” rule and the downstream “rank by realised ROI” are selection on the test set, the same bias we fixed in the trainer. Select on the new `include_in_ensemble` flag (validation-based), and use outsample ROI only as a report-only confirmation.
- **If you still want to batch** (for hard checkpoints), it is methodologically safe but operationally hazardous as-is — same-day batches overwrite each other’s models and log. Use the offset/subfolder convention in §5 if you go that way.

2. What changed in the code (V26.07)

Two additions land directly on your question:

1. `include_in_ensemble` column. Each model is flagged `1` when its **validation** AUC clears a floor (default 0.5, set with `-F`). This is the operational form of “ensemble every model above a sanity floor rather than cherry-pick a top-X” — and crucially the floor is on the honest insample holdout, not the outsample. The end-of-run summary prints how many cleared it.
2. **Incremental, crash-safe log.** The log CSV is appended row-by-row as models finish (header once), then overwritten with the sorted version at the end. A run that dies at epoch 900 keeps 1–899. You can `tail` or open the log mid-run and already see `val_auc_best` and `include_in_ensemble` for every finished model.

Together these mean the thing batching was really for — *getting actionable rankings before the run ends* — now happens continuously inside one run.

3. Why your current mid-training selection is the real risk

The cadence is harmless. The selection rule is not. Copying partial trees and keeping the ones whose **precision and recall both exceed 0.5 on the outsample** is selecting models on the very data you then use to judge the ensemble — identical in kind to the test-AUC ranking we removed from the trainer, and it inflates expected live performance the same way.

The downstream pipeline has the same trap. `batch_analysis.py` ranks models by **realised ROI on the outsample**, and the README frames a weak AUC-vs-ROI correlation as justification for ROI-based selection. But weak correlation only means AUC doesn't predict ROI — it does **not** mean outsample ROI predicts **future** ROI. Both are noisy measurements on the same period; selecting on outsample ROI just overfits the evaluation set in dollars instead of AUC.

Two more reasons to drop the precision/recall-at-0.5 gate specifically:

- **It is threshold-sensitive.** Precision and recall at the default 0.5 cut move around with class balance and the arbitrary cut point. Validation **AUC** is threshold-free and a cleaner gate — which is what `include_in_ensemble` uses.
- **It is a tiny, noisy sample.** “Both > 0.5, which is not often” means you are keeping the handful that happened to look good on one outsample — the textbook way to assemble a backtest that doesn't replicate live.

4. Recommended workflow

Keep one long run; select on validation; confirm on outsample.

1. **Run the full 1,000 in one go.** With live logging + MLflow there is no waiting penalty and no batching hazards. Let it run.
2. **Monitor live.** `tail -f` the log (or sort the MLflow runs by `val_auc`). The `include_in_ensemble` column tells you in real time which finished models are candidates.
3. **Form the prototype ensemble from `include_in_ensemble = 1`** — i.e. every model above the validation floor, not a hand-picked few. Given that validation barely predicts test (corr $\approx$ 0.19), breadth and variance-reduction are where the benefit is; aggressive selection mostly fits noise.
4. **Then run `dTREEPREDS` / `sports_roi` as a report-only check**, not a selector. Look at outsample ROI and the surprise-free subset to **understand** the ensemble, not to choose its members.
5. **Keep a final untouched holdout** (or use genuinely forward live games) for the chosen ensemble's honest number. Nothing that selected the models should appear in it.

Net effect on your routine: you still inspect models mid-run exactly as you like — but the keep/drop decision moves from “precision & recall > 0.5 on the outsample” to “`include_in_ensemble = 1`”, and the outsample becomes confirmation rather than the gate. Same habit, honest basis.

5. If you do batch — the two hazards and the fix

Batching is methodologically fine: `random.seed()` is called with no argument, so every process draws fresh entropy and separate batches produce genuinely different models (no duplication). But two operational hazards bite as the script stands:

Hazard	Why
Same-day overwrite	The model folder is named by DATE only (..._YYYYMMDD) and models are numbered 1..N per run. Two batches on the same day write to the same folder and the second overwrites the first's .pkl files and the log.
Number collisions	Each batch numbers its models 1..200, so model_7 from batch 1 and batch 2 are different models with the same name — merging the logs/pkls needs renumbering.
No global ranking	Each 200-batch ranks only its own 200; you get five local rankings, not one global one, unless you pool and re-rank.

Workable batching convention, if you want checkpoints:

- Give each batch its **own dated/numbered subfolder** (e.g. ..._run01/, ..._run02/) so nothing overwrites.
- Apply a **model-number offset** per batch (batch 2 → 201–400, etc.) when you copy pkls into the shared ensemble pool, so names stay unique.
- **Pool the per-batch logs and re-rank by `val\_auc\_best`** across the union. Because the validation split is the deterministic time-ordered tail of the same insample, **val_auc_best** is directly comparable across batches — so a pooled ranking is valid.

That said, given live logging now exists, the only thing batching buys you over one long run is a hard checkpoint every few days. If that is the goal, prefer larger chunks (say 250–500) with the subfolder + offset convention — not many small 200s.

6. One adjacent note

If a 1,000-model search genuinely takes 1–2 weeks on 40 features × 40k rows, most of that cost is the per-epoch hyperparameter search. Two safe levers if you want more models per unit time without touching methodology: lower **-f** (searches per fold) — with 40 features the marginal value of very deep search is small — and lean on **-v 1** (halving search). Neither changes the selection logic; they just get you breadth faster, which (per the corr ≈ 0.19 finding) is worth more here than an exhaustive search per model.

One line: keep the single long run (now live-logged and crash-safe), build the ensemble from include_in_ensemble = 1 on the validation holdout, and demote outsample precision/recall and ROI from selectors to confirmations.